works, how it’s different from other testing types, and why you should care about it. We’ll cover the technical implementation and show you from the ground up how to use relevant tools and integrate contract testing as part of your continuous integration (CI) pipelines. Our main goal is to convince you that contract testing is an essential part of your testing strategies, especially when you work within a microservices architecture.

As teams move from a monolithic application to a microservices architecture, they face new challenges when it comes to testing because the integration points between different services can increase exponentially. There is also the challenge of testing all the different microservices together in a dedicated test environment. Different teams can deploy changes at different times or at the same time, which can result in unexpected changes that can break an environment. Relying on traditional testing techniques won’t scale up well when applied to complex systems, so it’s now important for teams to know how contract testing can help ensure that every change you make doesn’t affect other teams negatively.

## 1.1 An overview of contract testing

In its simplest definition, *contract testing* is used to test that two systems have a shared understanding of expectations. For example, when a web application sends a `GET` request to a data service, the web application has expectations regarding how the data service should respond. Contract testing captures the interactions between these two systems, which are then stored in a contract that both systems must adhere to.

It’s similar to when you sign a contract in the real world. A great example is when we signed our contract with Manning. The contract details the expectations from our side, which is to deliver this book, and at the same time, it includes what is also expected from Manning. If changes need to be made to the contract, both parties will need to be notified, and changes will only be made once everyone approves. Contract testing is similar in this sense. If a data service suddenly responds with a set of data that is different from what a web application expects, this change will be flagged and reviewed by both systems before the change is deployed to production.